

RAPPORT DE STAGE

ROYAUME DU MAROC**Ministère de la Santé et de la Protection Sociale****ISTA HAY SALAM****Institut Spécialisé de Technologie Appliquée****RAPPORT DE STAGE DE FIN D'ANNÉE****Filière : Développement Digital — 2ème Année****Réalisé par :****Milad DLOU****Organisme d'accueil :****Direction de l'Épidémiologie et de Lutte Contre les Maladies (DELM)**

71, Avenue Ibn Sina — Agdal, Rabat

Responsable de stage :

M. Haroun

Période du stage :

Du 01 Mars 2026 au 31 Mars 2026

Année scolaire : 2025 – 2026

REMERCIEMENTS

La réalisation de ce rapport de stage n'aurait pas été possible sans le concours de nombreuses personnes à qui je souhaite exprimer toute ma reconnaissance.

Je tiens tout d'abord à adresser mes sincères remerciements à **M. Haroun**, responsable de stage au sein de la **Direction de l'Épidémiologie et de Lutte Contre les Maladies (DELM)**, pour l'accueil chaleureux qu'il m'a réservé, sa disponibilité constante malgré la distance, ses conseils avisés et son suivi rigoureux tout au long de cette expérience. Sa guidance a été un appui précieux dans l'accomplissement de ce projet.

Je remercie également l'ensemble du personnel de la **DELM** pour leur sympathie et leur volonté de partager leurs expériences professionnelles, qui m'ont permis de mieux appréhender le contexte institutionnel dans lequel s'inscrit ce stage.

J'exprime ma gratitude à l'équipe pédagogique de l'**ISTA HAY SALAM**, et en particulier aux formateurs de la filière **Développement Digital**, pour la qualité de la formation dispensée et les compétences techniques transmises, qui ont constitué le socle indispensable à la réalisation de ce projet.

Enfin, je remercie ma famille pour son soutien moral et sa confiance tout au long de cette période.

TABLE DES MATIÈRES

1. Introduction générale
2. Présentation de l'organisme d'accueil
 - 2.1 Présentation générale de la DELM
 - 2.2 Missions et attributions
 - 2.3 Organisation interne
 - 2.4 Environnement informatique
3. Déroulement du stage
 - 3.1 Conditions et organisation
 - 3.2 Planning des activités
 - 3.3 Missions confiées
4. Présentation du projet : RendezVousApp
 - 4.1 Contexte et problématique
 - 4.2 Objectifs du projet
5. Analyse et conception

- 5.1 Analyse des besoins
- 5.2 Acteurs et cas d'utilisation
- 5.3 Architecture de l'application
- 5.4 Conception de la base de données

6. Technologies et outils utilisés

- 6.1 Technologies backend
- 6.2 Technologies frontend
- 6.3 Outils de développement

7. Réalisation

- 7.1 Backend — API REST avec Laravel
- 7.2 Frontend — Interface React.js
- 7.3 Présentation des interfaces

8. Difficultés rencontrées et solutions

9. Conclusion et bilan

10. Bibliographie et webographie

11. Annexes

LISTE DES TABLEAUX ET FIGURES

N°	Intitulé	Page
Tableau 1	Planning hebdomadaire du stage	6
Tableau 2	Langages et technologies utilisés à la DELM	5
Tableau 3	Besoins fonctionnels par acteur	9
Tableau 4	Technologies backend	14
Tableau 5	Technologies frontend	15
Tableau 6	Routes de l'API REST	17
Tableau 7	Difficultés rencontrées et solutions	20
Figure 1	Organigramme simplifié de la DELM	5
Figure 2	Architecture générale de l'application	10
Figure 3	Schéma Entité-Association	12
Figure 4	Diagramme de cas d'utilisation	9

LISTE DES ABRÉVIATIONS

Abréviation	Signification
DELM	Direction de l'Épidémiologie et de Lutte Contre les Maladies
ISTA	Institut Spécialisé de Technologie Appliquée
DD	Développement Digital
API	Application Programming Interface
REST	Representational State Transfer
SPA	Single Page Application
MVC	Modèle-Vue-Contrôleur
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
SQL	Structured Query Language
FK	Foreign Key (Clé étrangère)
RDV	Rendez-vous

1. INTRODUCTION GÉNÉRALE

Dans le cadre de la formation en **Développement Digital (2ème année)** dispensée à l'**ISTA HAY SALAM**, les étudiants sont tenus d'effectuer un stage professionnel à l'issue de chaque année académique. Ce stage constitue un moment fondamental dans le cursus de formation : il représente le pont entre la théorie apprise en salle et la pratique réelle du métier de développeur.

C'est dans ce cadre que j'ai eu l'opportunité d'effectuer un stage d'un mois — du **1er mars 2026** au **31 mars 2026** — au sein de la **Direction de l'Épidémiologie et de Lutte Contre les Maladies (DELM)**, une institution publique de santé rattachée au **Ministère de la Santé et de la Protection Sociale du Maroc**,

dont le siège est situé au **71, Avenue Ibn Sina, Agdal, Rabat**. Ce stage a été réalisé **à distance**, sous la supervision de **M. Haroun**, responsable de stage.

La mission principale qui m'a été confiée consistait à **concevoir et développer une application web de gestion des rendez-vous médicaux**, baptisée **RendezVousApp**. Ce projet s'inscrit directement dans la mission de la DELM, qui œuvre au quotidien pour l'amélioration de l'accès aux soins et la modernisation des services de santé au Maroc. En facilitant la prise de rendez-vous entre patients et médecins, cette application vise à réduire les délais d'attente, à fluidifier la gestion des agendas médicaux et à améliorer l'expérience des usagers du système de santé.

Ce rapport présente de façon structurée le travail accompli pendant cette période. Il est organisé en plusieurs chapitres : après une présentation de l'organisme d'accueil et du déroulement du stage, nous détaillerons les phases d'analyse, de conception et de réalisation technique du projet, avant de conclure sur le bilan personnel et professionnel de cette expérience.

2. PRÉSENTATION DE L'ORGANISME D'ACCUEIL

2.1 Présentation générale de la DELM

La **Direction de l'Épidémiologie et de Lutte Contre les Maladies (DELM)** est une direction centrale du **Ministère de la Santé et de la Protection Sociale du Maroc**. Elle est implantée au **71, Avenue Ibn Sina, Agdal, Rabat**, au cœur du quartier administratif qui abrite l'essentiel des institutions gouvernementales marocaines.

Créée dans le but de centraliser et de coordonner les efforts nationaux en matière de surveillance épidémiologique et de prévention des maladies, la DELM constitue un acteur incontournable de la santé publique au Maroc. Elle travaille en étroite collaboration avec les directions régionales de santé, les hôpitaux publics, les partenaires internationaux (OMS, UNICEF) et les instituts de recherche.

Son champ d'action couvre l'ensemble du territoire marocain et touche à des domaines aussi variés que la surveillance des maladies infectieuses, la gestion des vaccinations nationales, la lutte contre les maladies chroniques, ou encore la réponse aux urgences sanitaires.

2.2 Missions et attributions

La DELM exerce des missions fondamentales dans le domaine de la santé publique :

- **Surveillance épidémiologique** : Collecte, analyse et interprétation des données sanitaires sur les maladies transmissibles et non transmissibles à l'échelle nationale.
- **Lutte contre les maladies transmissibles** : Coordination des programmes de lutte contre des pathologies telles que la tuberculose, les maladies à prévention vaccinale, le VIH/SIDA, les hépatites virales, etc.
- **Programme National d'Immunisation (PNI)** : Planification et suivi des campagnes de vaccination à l'échelle nationale.
- **Prévention des maladies non transmissibles** : Mise en place de stratégies de prévention du diabète, des maladies cardiovasculaires, des cancers.
- **Gestion des alertes sanitaires** : Détection précoce et réponse rapide aux épidémies et aux situations d'urgence sanitaire.
- **Systèmes d'information sanitaire** : Développement et maintenance des outils informatiques permettant la collecte et le traitement des données de santé.
- **Coordination et partenariat** : Collaboration avec les organismes nationaux et internationaux pour l'harmonisation des pratiques en santé publique.

2.3 Organisation interne

La DELM est organisée en plusieurs divisions et services dont les attributions couvrent l'ensemble de son périmètre d'intervention :

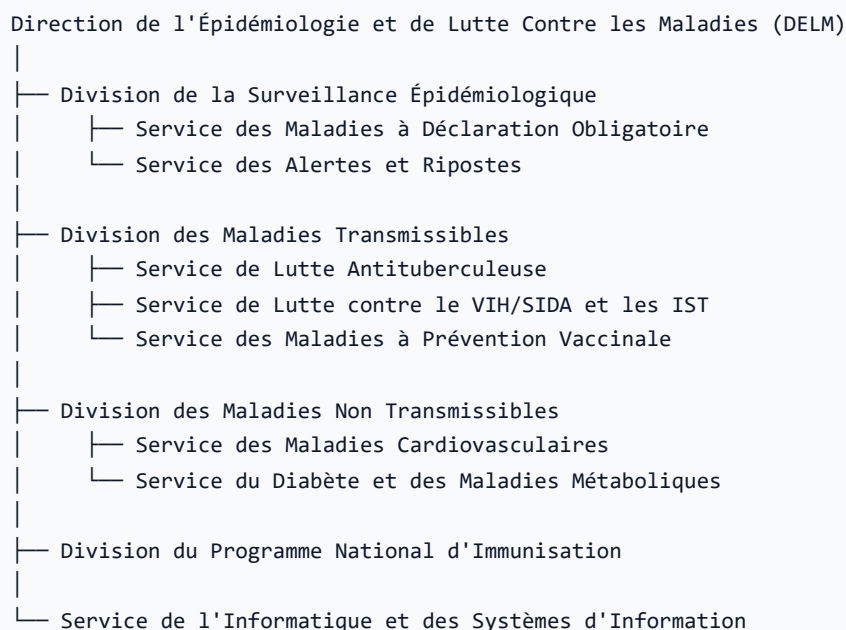


Figure 1 — Organigramme simplifié de la DELM

2.4 Environnement informatique

La DELM dispose d'un service informatique dédié qui assure le développement et la maintenance des systèmes d'information de santé. Dans le cadre de ses activités de développement, les technologies suivantes sont utilisées :

Tableau 2 — Langages et technologies utilisés à la DELM

Domaine	Technologie	Utilisation
Développement backend	PHP	Développement d'applications web métier, APIs
Développement backend	Python	Analyse de données épidémiologiques, scripts de traitement
Développement backend	Java	Applications entreprise, systèmes d'information hospitaliers
Bases de données	MySQL / Oracle	Stockage des données de santé
Serveurs	Linux / Apache	Hébergement des applications internes

C'est dans cet environnement technologique que s'est inscrit mon stage, avec une mission de développement web PHP (Laravel) et JavaScript (React), langages parfaitement cohérents avec l'écosystème technique de la direction.

3. DÉROULEMENT DU STAGE

3.1 Conditions et organisation

Ce stage d'une durée d'un mois (**du 1er mars 2026 au 31 mars 2026**) a été effectué **entièrement à distance**, en accord avec la DELM et conformément aux modalités convenues dès le début du stage. Cette organisation en télétravail, de plus en plus répandue dans le secteur informatique, m'a permis de développer des compétences essentielles en matière d'autonomie, de rigueur dans l'organisation personnelle, et de communication à distance.

Le suivi du stage était assuré par **M. Haroun**, responsable de stage, qui effectuait des points réguliers pour évaluer l'avancement du projet, donner des orientations et valider les livrables. Ces échanges se faisaient via des outils de communication à distance (messaging, visioconférence).

La gestion du code source était assurée via **Git**, permettant le suivi des versions et le partage du code avec le tuteur.

3.2 Planning des activités

Tableau 1 — Planning hebdomadaire du stage

Semaine	Dates	Activités
Semaine 1	01 – 07 Mars 2026	Prise de contact, analyse du contexte, étude des besoins, conception de la base de données, choix des technologies
Semaine 2	08 – 15 Mars 2026	Mise en place du projet Laravel, création des modèles, migrations, configuration de l'authentification Sanctum
Semaine 3	16 – 23 Mars 2026	Développement des contrôleurs API (Auth, RendezVous, Médecin, Secrétaire, Créneaux, Services), tests API avec Postman
Semaine 4	24 – 31 Mars 2026	Développement du frontend React.js, intégration API, gestion du routing et des rôles, tests finaux, rédaction du rapport

3.3 Missions confiées

Les missions qui m'ont été confiées au cours de ce stage peuvent se résumer ainsi :

1. **Analyse des besoins** du projet RendezVousApp en concertation avec M. Haroun.
2. **Conception** de l'architecture technique et du modèle de données.
3. **Développement backend** : création d'une API REST sécurisée avec Laravel et Sanctum.
4. **Développement frontend** : réalisation d'une interface utilisateur moderne avec React.js.
5. **Tests et validation** de l'application.
6. **Rédaction** de la documentation technique et du présent rapport.

4. PRÉSENTATION DU PROJET : RendezVousApp

4.1 Contexte et problématique

Dans le secteur de la santé, la gestion des rendez-vous représente un enjeu organisationnel majeur. Traditionnellement, cette gestion repose sur des registres papier ou des agendas téléphoniques qui engendrent de nombreux problèmes :

- **Double réservation** de créneaux horaires

- **Temps d'attente** prolongés pour les patients
- **Absence de traçabilité** des rendez-vous annulés
- **Difficultés de communication** entre les patients et le personnel médical
- **Perte d'informations** liée aux supports non numériques

Face à cette problématique, la DELM m'a confié le développement d'une application web visant à **dématérialiser et automatiser la gestion des rendez-vous médicaux**. Ce type de solution est particulièrement pertinent dans le contexte de la transition numérique que vit le système de santé marocain, soutenu par des initiatives gouvernementales telles que la **stratégie nationale de santé numérique**.

4.2 Objectifs du projet

Le projet **RendezVousApp** vise à atteindre les objectifs suivants :

Objectifs fonctionnels :

- Offrir aux patients un espace en ligne pour prendre et gérer leurs rendez-vous
- Permettre aux médecins de gérer leurs disponibilités et leurs services
- Donner aux secrétaires un outil d'administration des dossiers patients
- Assurer une communication fluide entre les différents acteurs

Objectifs techniques :

- Développer une API REST robuste et sécurisée
- Créer une interface utilisateur moderne, réactive et ergonomique
- Implémenter un système d'authentification et de contrôle des accès par rôle
- Garantir l'intégrité des données (pas de double réservation, cohérence des créneaux)

Objectifs pédagogiques :

- Mettre en pratique les compétences acquises à l'ISTA (PHP, JavaScript, bases de données)
- Découvrir des frameworks professionnels (Laravel, React.js)
- Travailler en autonomie sur un projet complet de A à Z

5. ANALYSE ET CONCEPTION

5.1 Analyse des besoins

Besoins fonctionnels

L'application doit prendre en charge trois types d'acteurs, chacun disposant d'un espace dédié et de fonctionnalités spécifiques :

Tableau 3 — Besoins fonctionnels par acteur

Acteur	Fonctionnalités
Patient	Créer un compte · Se connecter/déconnecter · Consulter les créneaux disponibles · Prendre un rendez-vous · Annuler un rendez-vous · Consulter l'historique de ses RDV · Gérer son profil
Médecin	Se connecter/déconnecter · Créer des créneaux horaires · Modifier/supprimer des créneaux · Gérer ses services médicaux · Créer un compte secrétaire · Consulter ses rendez-vous · Gérer son profil
Secrétaire	Se connecter/déconnecter · Consulter la liste des patients · Consulter la liste des rendez-vous · Gérer son profil

Besoins non fonctionnels

- **Sécurité** : Authentification par token (pas de cookies), mots de passe hachés (bcrypt), routes protégées par rôle.
- **Performance** : Réponses API rapides, chargement optimisé du frontend (Vite).
- **Disponibilité** : Application accessible depuis n'importe quel navigateur moderne.
- **Maintenabilité** : Code structuré (MVC, composants React), séparation claire backend/frontend.
- **Ergonomie** : Interface intuitive, navigation fluide, retours utilisateur immédiats.

5.2 Acteurs et cas d'utilisation

Les interactions entre les acteurs et le système sont représentées par le diagramme de cas d'utilisation ci-dessous :

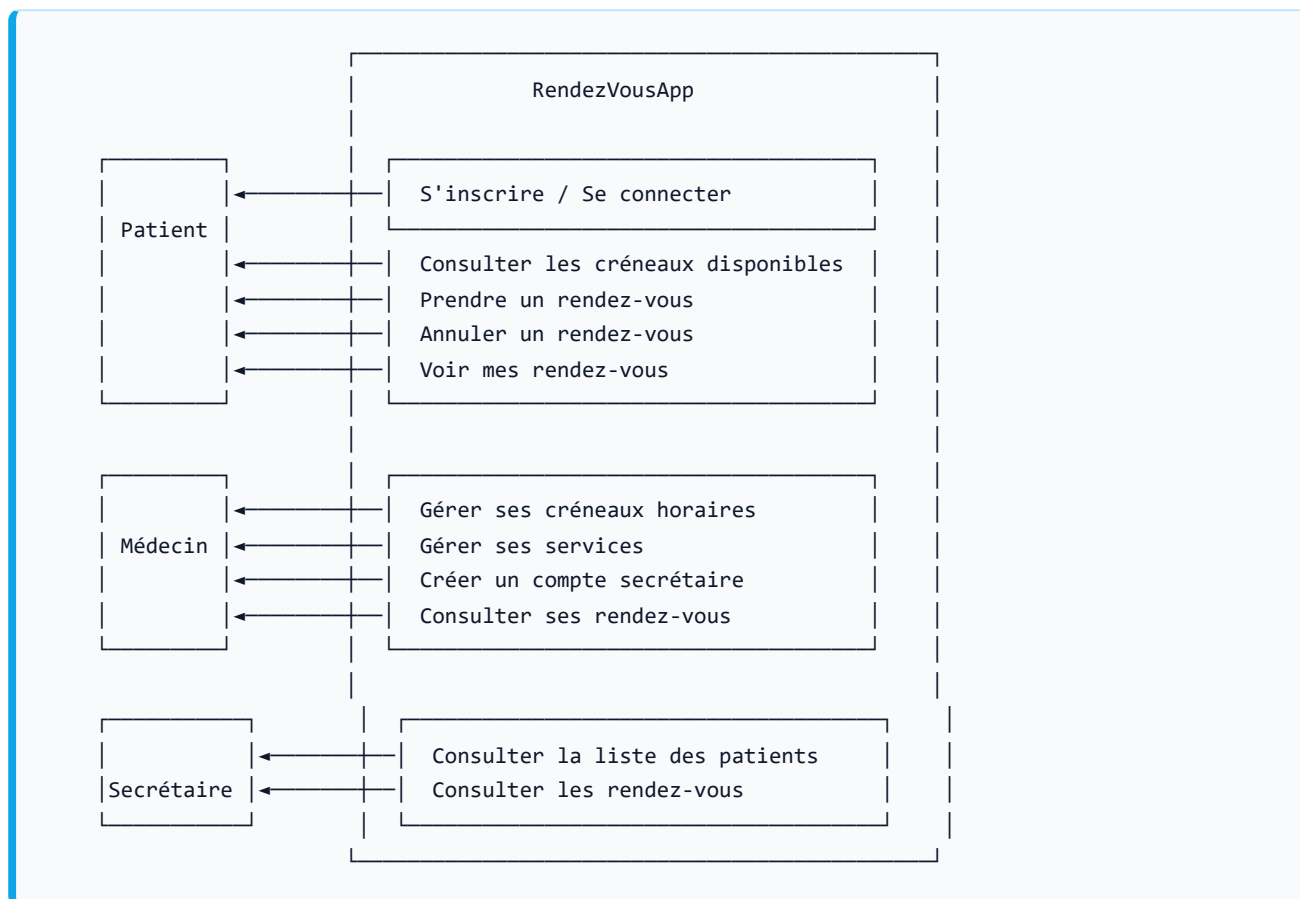


Figure 4 — Diagramme de cas d'utilisation

5.3 Architecture de l'application

L'application adopte une architecture **full-stack découplée** (découplage complet entre backend et frontend), suivant le modèle **Client-Serveur** :

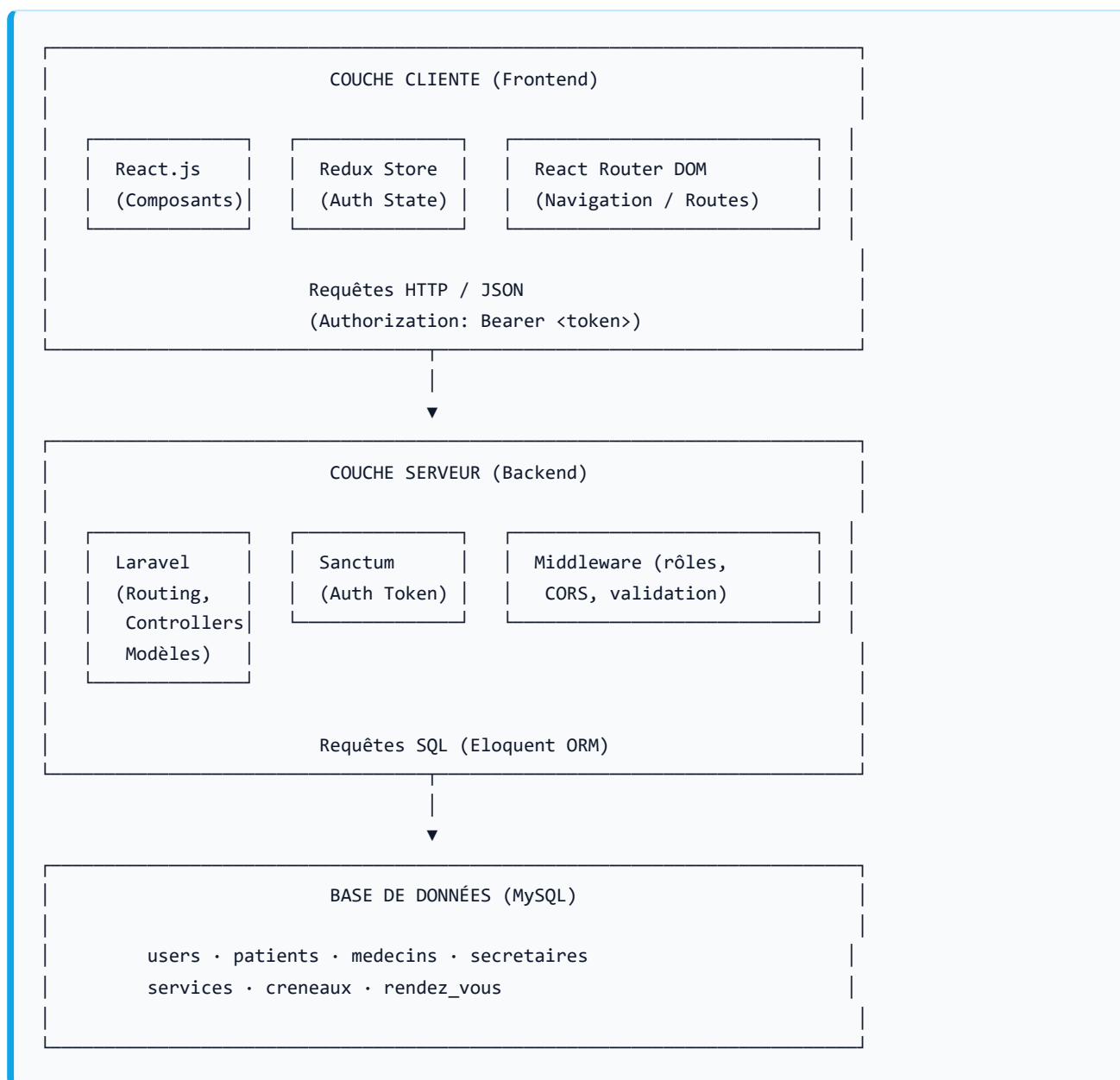


Figure 2 — Architecture générale de l'application

Flux de fonctionnement :

1. L'utilisateur interagit avec l'interface React (SPA).
2. React envoie des requêtes HTTP à l'API Laravel avec le token d'authentification dans l'en-tête.
3. Laravel valide le token via Sanctum, vérifie les autorisations et traite la requête.
4. Eloquent ORM communique avec MySQL pour lire ou écrire les données.
5. Laravel retourne une réponse JSON que React affiche à l'utilisateur.

5.4 Conception de la base de données

Modèle Entité-Association

La base de données est composée de **7 tables principales**. Voici les entités et leurs relations :

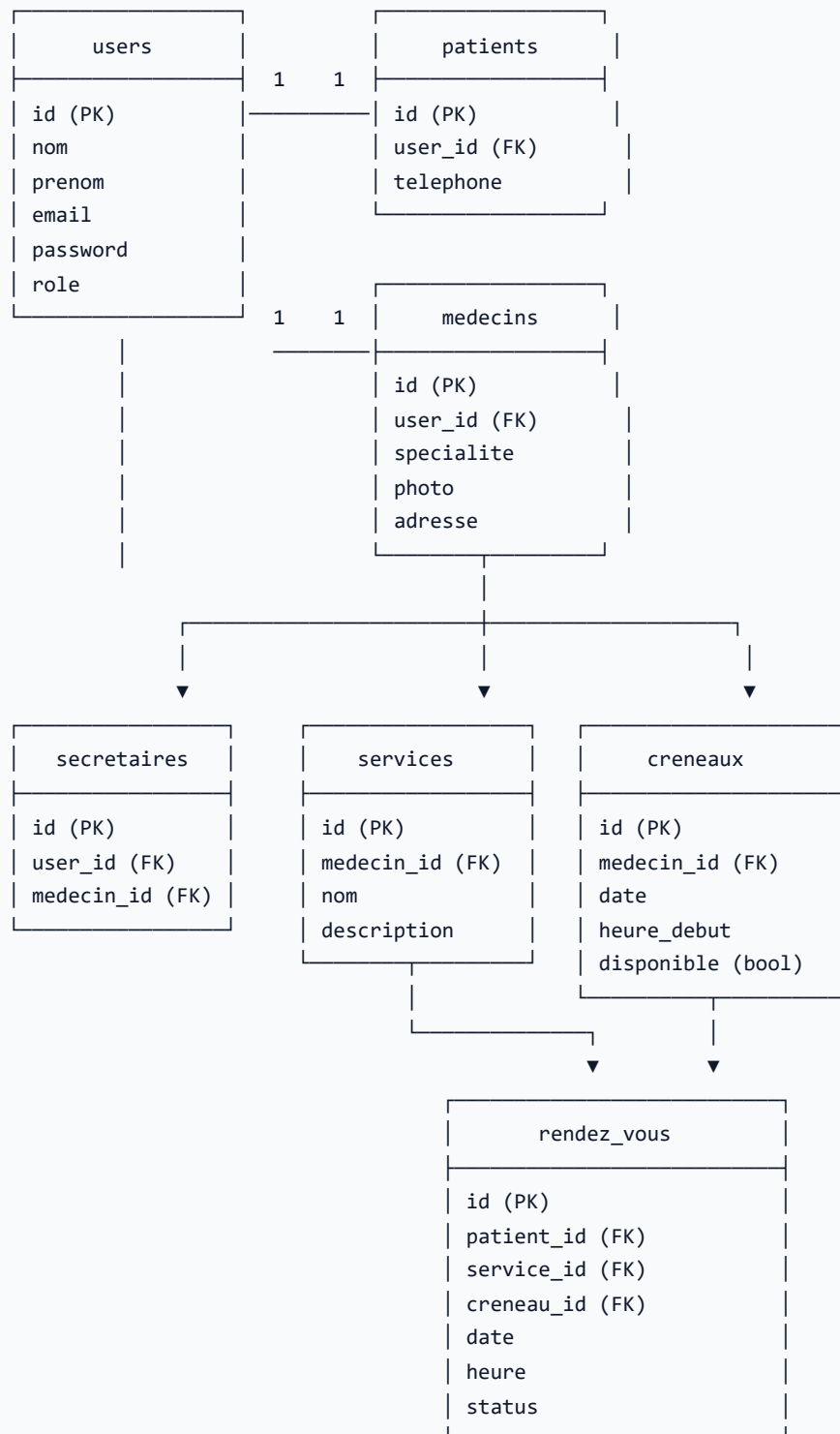


Figure 3 — Schéma Entité-Association

Description détaillée des tables

Table **users** — Entité de base commune à tous les utilisateurs

Stocke les informations d'authentification et le rôle de chaque utilisateur (patient, medecin, secretaire).

Champ	Type	Description
id	INT (PK)	Identifiant unique
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
email	VARCHAR (unique)	Adresse email (identifiant de connexion)
password	VARCHAR	Mot de passe haché (bcrypt)
role	ENUM	Rôle : patient , medecin , secretaire

Table **patients** — Profil spécifique du patient

Champ	Type	Description
id	INT (PK)	Identifiant unique
user_id	INT (FK → users)	Référence vers l'utilisateur
telephone	VARCHAR	Numéro de téléphone

Table **medecins** — Profil spécifique du médecin

Champ	Type	Description
id	INT (PK)	Identifiant unique
user_id	INT (FK → users)	Référence vers l'utilisateur
specialite	VARCHAR	Spécialité médicale
photo	VARCHAR	Chemin vers la photo de profil
adresse	VARCHAR	Adresse du cabinet

Table secretaires — Profil secrétaire, rattaché à un médecin

Champ	Type	Description
id	INT (PK)	Identifiant unique
user_id	INT (FK → users)	Référence utilisateur
medecin_id	INT (FK → medecins)	Médecin auquel la secrétaire est rattachée

Table services — Services médicaux proposés par le médecin

Champ	Type	Description
id	INT (PK)	Identifiant unique
medecin_id	INT (FK → medecins)	Médecin propriétaire du service
nom	VARCHAR	Intitulé du service (ex : Consultation générale)
description	TEXT	Description détaillée

Table creneaux — Créneaux horaires disponibles

Champ	Type	Description
id	INT (PK)	Identifiant unique
medecin_id	INT (FK → medecins)	Médecin auquel appartient le créneau
date	DATE	Date du créneau
heure_debut	TIME	Heure de début
disponible	BOOLEAN	Disponibilité (true = libre, false = pris)

Table rendez_vous — Table centrale, cœur du système

Champ	Type	Description
id	INT (PK)	Identifiant unique
patient_id	INT (FK → patients)	Patient concerné
service_id	INT (FK → services)	Service demandé
creneau_id	INT (FK → creneaux)	Créneau réservé
date	DATE	Date du rendez-vous
heure	TIME	Heure du rendez-vous
status	ENUM	État : en_attente , confirme , annule

6. TECHNOLOGIES ET OUTILS UTILISÉS

6.1 Technologies backend

Tableau 4 — Technologies backend

Technologie	Version	Rôle dans le projet
PHP	8.x	Langage de programmation serveur
Laravel	11	Framework MVC : routing, ORM, validations, middleware, gestion des erreurs
Laravel Sanctum	—	Authentification API par tokens personnels (API Token Authentication)
Eloquent ORM	—	Mapping objet-relationnel, relations (hasOne, hasMany, belongsTo)
MySQL	8.x	Base de données relationnelle
Composer	—	Gestionnaire de dépendances PHP

Pourquoi Laravel ? Laravel est le framework PHP le plus utilisé dans le développement web professionnel. Il est particulièrement adapté au développement d'APIs REST grâce à son système de routing expressif, son ORM Eloquent puissant et sa bibliothèque Sanctum pour la gestion des tokens d'authentification. Il est parfaitement cohérent avec l'environnement PHP de la DELM.

6.2 Technologies frontend

Tableau 5 — Technologies frontend

Technologie	Version	Rôle dans le projet
React.js	18	Bibliothèque JavaScript pour la construction de l'interface utilisateur (composants réutilisables)
Redux Toolkit	—	Gestion de l'état global de l'application (token, données utilisateur)
React Router DOM	v6	Gestion de la navigation et des routes côté client (SPA)
Vite	5.x	Outil de build et serveur de développement ultra-rapide
Axios / Fetch	—	Envoi des requêtes HTTP vers l'API avec injection automatique du token
Node.js / npm	—	Environnement d'exécution JavaScript et gestionnaire de paquets

6.3 Outils de développement

Outil	Utilisation
Visual Studio Code	Éditeur de code principal
Postman	Test et validation des endpoints de l'API REST
Git / GitHub	Gestion des versions et partage du code avec le tuteur
XAMPP / Laragon	Environnement de développement local (Apache, MySQL, PHP)
PhpMyAdmin	Administration de la base de données MySQL
Navigateur (Chrome/Firefox)	Test de l'interface utilisateur

7. RÉALISATION

7.1 Backend — API REST avec Laravel

Structure du projet Laravel

Le projet backend suit rigoureusement l'architecture **MVC** (Modèle-Vue-Contrôleur) de Laravel, adaptée au développement d'API (sans vues HTML, toutes les réponses sont en JSON) :

```
rendez-vous-backend/  
├── app/  
│   ├── Http/  
│   │   └── Controllers/  
│   │       ├── AuthController.php           ← Inscription, connexion, déconnexion  
│   │       ├── RendezVousController.php     ← Gestion des rendez-vous  
│   │       ├── MedecinController.php        ← Gestion des médecins  
│   │       ├── SecretaireController.php      ← Gestion des secrétaires  
│   │       ├── PatientController.php         ← Gestion des patients  
│   │       ├── CreneauController.php         ← Gestion des créneaux  
│   │       ├── ServiceController.php         ← Gestion des services  
│   │       └── emailController.php           ← Envoi d'emails  
│   └── Models/  
│       ├── User.php  
│       ├── Patient.php  
│       ├── Medecin.php  
│       ├── Secretaire.php  
│       ├── Service.php  
│       ├── Creneau.php  
│       └── RendezVous.php  
├── database/  
│   └── migrations/                          ← Scripts de création des tables  
└── routes/  
    └── api.php                              ← Définition des routes API
```

Tableau 6 — Routes principales de l'API REST

Méthode	Route	Contrôleur	Description	Auth
POST	<code>/api/register</code>	AuthController	Inscription patient	Non
POST	<code>/api/login</code>	AuthController	Connexion (retourne token)	Non
POST	<code>/api/logout</code>	AuthController	Déconnexion	Oui
POST	<code>/api/rendezvous/prendre</code>	RendezVousController	Prendre un RDV	Oui (patient)
DELETE	<code>/api/rendezvous/{id}</code>	RendezVousController	Annuler un RDV	Oui (patient)
GET	<code>/api/rendezvous/mes</code>	RendezVousController	Mes rendez-vous	Oui (patient)
POST	<code>/api/creneaux</code>	CreneauController	Créer un créneau	Oui (médecin)
GET	<code>/api/creneaux</code>	CreneauController	Lister les créneaux	Oui
DELETE	<code>/api/creneaux/{id}</code>	CreneauController	Supprimer un créneau	Oui (médecin)
GET	<code>/api/services</code>	ServiceController	Lister les services	Oui
POST	<code>/api/services</code>	ServiceController	Créer un service	Oui (médecin)
POST	<code>/api/medecin/creer</code>	MedecinController	Créer un médecin	Non
GET	<code>/api/medecin/profil</code>	MedecinController	Profil médecin	Oui
POST	<code>/api/secretaire/creer</code>	SecretaireController	Créer secrétaire	Oui (médecin)
GET	<code>/api/patients</code>	PatientController	Lister les patients	Oui (secrétaire)

Fonctionnement de l'authentification

L'authentification est gérée par **Laravel Sanctum**, qui attribue à chaque connexion un **token d'accès personnel** unique. Ce token est transmis par le frontend dans l'en-tête HTTP de chaque requête :

```
Authorization: Bearer <token>
```

Lors de l'inscription, le système crée simultanément un enregistrement dans la table `users` et un profil dans la table correspondant au rôle (ici `patients`). Le mot de passe est systématiquement haché avec `bcrypt` avant stockage.

Lors de la déconnexion, le token courant est supprimé de la base de données, invalidant toute tentative d'utilisation ultérieure.

Logique de prise de rendez-vous

La prise de rendez-vous est le cœur fonctionnel de l'application. La logique implémentée dans `RendezVousController::prendre()` garantit l'intégrité des données :

1. Récupération du patient à partir du token (utilisateur connecté).
2. Vérification de l'existence du créneau demandé.
3. Vérification que le créneau est bien disponible (`disponible = true`).
4. Création du rendez-vous avec le statut `en_attente` .
5. Mise à jour du créneau : `disponible = false` (le créneau n'est plus visible pour d'autres patients).

Cette logique de validation côté serveur garantit qu'un créneau ne peut jamais être réservé deux fois, même en cas de requêtes simultanées.

7.2 Frontend — Interface React.js

Structure du projet React

Le frontend est organisé selon les conventions React modernes :

```

rendez-vous-frontend/
├─ src/
│  ├─ pages/
│  │  ├─ Login.jsx          ← Page de connexion
│  │  ├─ Register.jsx       ← Page d'inscription
│  │  ├─ Dashboard.jsx      ← Tableau de bord patient
│  │  ├─ medecin.jsx        ← Tableau de bord médecin
│  │  └─ secretaire.jsx     ← Tableau de bord secrétaire
│  │
│  ├─ components/
│  │  ├─ prendreRDV.jsx     ← Formulaire de prise de RDV
│  │  ├─ mesRDV.jsx         ← Liste des RDV du patient
│  │  ├─ listerRDV.jsx      ← Liste des RDV (médecin/secrétaire)
│  │  ├─ listePatient.jsx   ← Liste des patients (secrétaire)
│  │  ├─ calendrier.jsx     ← Vue calendrier des créneaux
│  │  ├─ GestionCreneau.jsx ← CRUD créneaux (médecin)
│  │  ├─ GererService.jsx   ← CRUD services (médecin)
│  │  ├─ CreerSecrétaire.jsx ← Création compte secrétaire
│  │  ├─ profil.jsx         ← Profil patient
│  │  ├─ ProfilMedecin.jsx  ← Profil médecin
│  │  └─ profilSecrétaire.jsx ← Profil secrétaire
│  │
│  ├─ store/
│  │  └─ slices/
│  │     ├─ AuthSlice.js    ← Slice Redux (auth state)
│  │     └─ index.js        ← Configuration du store Redux
│  │
│  ├─ Routes/
│  │  └─ protectedRoutes.jsx ← HOC de protection des routes par rôle
│  │
│  ├─ services/
│  │  ├─ api.jsx            ← Configuration Axios (token, base URL)
│  │  └─ secretaire.jsx     ← Appels API secrétaire
│  │
│  ├─ App.jsx              ← Composant racine, configuration des routes
│  └─ main.jsx              ← Point d'entrée React

```

Gestion de l'état global avec Redux

Le **AuthSlice** Redux centralise toutes les informations liées à l'utilisateur connecté :

- Le token d'accès
- Les données de l'utilisateur (nom, prénom, rôle, id)
- L'état de connexion (authentifié / non authentifié)

Ces informations sont persistées dans le **localStorage** du navigateur, permettant à l'utilisateur de rester connecté après un rechargement de page.

Système de routes protégées

Le composant `ProtectedRoutes` agit comme un **garde-route** : avant d'afficher un composant, il vérifie dans le store Redux que l'utilisateur est bien authentifié **et** qu'il possède le rôle requis. Dans le cas contraire, il redirige automatiquement vers la page de connexion.

```
/register      → accessible à tous
/login         → accessible à tous
/dashboard     → réservé au rôle "patient"
/dashboard/medecin → réservé au rôle "medecin"
/dashboard/secretaire → réservé au rôle "secretaire"
/*            → redirection vers /login
```

7.3 Présentation des interfaces

Interface 1 — Page de connexion (`/login`)

La page de connexion présente un formulaire épuré demandant l'adresse email et le mot de passe. Après validation, le token reçu est stocké dans le store Redux et l'utilisateur est redirigé vers son tableau de bord en fonction de son rôle.

Interface 2 — Tableau de bord Patient (`/dashboard`)

Le tableau de bord patient centralise toutes les fonctionnalités accessibles au patient :

- Un panneau **"Prendre un rendez-vous"** (`prendreRDV.jsx`) affiche les services disponibles et les créneaux libres. Le patient sélectionne un créneau et confirme sa réservation.
- Un panneau **"Mes rendez-vous"** (`mesRDV.jsx`) liste tous les RDV passés et à venir avec leur statut, et permet l'annulation.
- Un panneau **"Calendrier"** (`calendrier.jsx`) offre une vue temporelle des disponibilités.
- Un accès au **profil** pour modifier ses informations personnelles.

Interface 3 — Tableau de bord Médecin (`/dashboard/medecin`)

Le tableau de bord médecin offre des outils de gestion :

- **Gestion des créneaux** (`GestionCreneau.jsx`) : ajout, suppression et consultation des créneaux horaires.

- **Gestion des services** (`GenerService.jsx`) : création et modification des services médicaux proposés.
- **Création d'une secrétaire** (`CreerSecretaire.jsx`) : formulaire de création d'un compte secrétaire rattaché au médecin.
- **Liste des rendez-vous** (`listeRDV.jsx`) : vue d'ensemble des RDV avec filtres possibles.
- **Profil médecin** (`ProfilMedecin.jsx`) : mise à jour des informations (spécialité, adresse, photo).

Interface 4 — Tableau de bord Secrétaire (`/dashboard/secretaire`)

La secrétaire dispose d'un espace de consultation et de gestion administrative :

- **Liste des patients** (`listePatient.jsx`) : annuaire des patients inscrits.
- **Liste des rendez-vous** (`listeRDV.jsx`) : vue complète des rendez-vous.
- **Profil** (`profilSecretaire.jsx`) : gestion des informations personnelles.

8. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Tableau 7 — Difficultés rencontrées et solutions apportées

#	Difficulté	Cause	Solution adoptée
1	Problème de CORS lors des appels du frontend vers l'API	Le navigateur bloque les requêtes cross-origin (port 5173 vers port 8000)	Configuration du middleware CORS dans Laravel (<code>config/cors.php</code>) pour autoriser le domaine frontend
2	Double réservation d'un créneau	Deux patients pouvant accéder simultanément	Vérification de disponibilité côté serveur avant chaque création de RDV, mise à jour immédiate du statut <code>disponible</code>
3	Gestion des rôles côté frontend	Besoin de restreindre l'accès aux pages selon le rôle	Création d'un composant <code>ProtectedRoutes</code> utilisant le store Redux pour vérifier le rôle avant le rendu
4	Persistance du token après rechargement	Redux réinitialise l'état à chaque rechargement	Synchronisation du store Redux avec le <code>localStorage</code> du navigateur
5	Authentification Sanctum	Token à inclure dans chaque requête HTTP	Centralisation de toutes les requêtes dans <code>api.js</code> (Axios instance) avec injection automatique du token
6	Organisation du travail à distance	Absence de contact direct avec le tuteur	Utilisation de Git pour versionner le code, points réguliers avec M. Haroun, documentation continue
7	Annulation de RDV sans libérer le créneau	Oubli de remettre <code>disponible = true</code>	Ajout de la mise à jour du créneau dans la méthode <code>annuler()</code> du <code>RendezVousController</code>

Ces difficultés, bien que représentant des obstacles dans le déroulement du projet, ont été autant d'opportunités d'apprentissage. Elles m'ont permis de développer ma capacité à diagnostiquer des problèmes techniques et à y apporter des solutions appropriées de façon autonome.

9. CONCLUSION ET BILAN

Bilan technique

Au terme de ce stage d'un mois au sein de la **Direction de l'Épidémiologie et de Lutte Contre les Maladies**, j'ai mené à bien la conception et le développement d'une application web complète de **gestion des rendez-vous médicaux** — **RendezVousApp**.

L'application livrée est une solution **full-stack fonctionnelle** qui répond à l'ensemble des besoins exprimés :

- Les patients peuvent s'inscrire, se connecter, prendre et annuler des rendez-vous.
- Les médecins gèrent leurs créneaux, leurs services et leur secrétaire depuis un tableau de bord dédié.
- Les secrétaires consultent les listes de patients et de rendez-vous.
- Le système d'authentification par token garantit la sécurité des accès.
- L'architecture découplée (Laravel + React) assure la maintenabilité et l'évolutivité de la solution.

Bilan personnel et professionnel

Ce stage a représenté une expérience formatrice à plusieurs niveaux :

Sur le plan technique, j'ai approfondi ma maîtrise de **PHP** et découvert le framework **Laravel**, largement utilisé dans le secteur professionnel. J'ai également progressé en **JavaScript** avec **React.js** et **Redux**, et consolidé mes compétences en **conception de bases de données relationnelles**.

Sur le plan professionnel, travailler à distance pour une institution publique de l'envergure de la DELM m'a appris à structurer mon travail de manière rigoureuse, à respecter des délais, à communiquer efficacement avec un responsable, et à prendre des initiatives face aux obstacles techniques.

Sur le plan personnel, ce projet m'a confirmé mon intérêt et ma motivation pour le développement web full-stack, et m'a donné confiance en ma capacité à mener un projet informatique complet de façon autonome.

Perspectives d'évolution

L'application **RendezVousApp** pourrait être enrichie à l'avenir par les fonctionnalités suivantes :

- **Notifications par email** : envoi automatique d'une confirmation de RDV au patient et au médecin (module `emailController.php` déjà présent dans le projet).
- **Application mobile** : développement d'une version React Native pour smartphones.
- **Tableau de bord statistique** : visualisation des données (taux d'occupation, nombre de RDV par service, etc.) à destination des médecins et du personnel administratif.
- **Système de rappels** : notifications automatiques 24h avant chaque rendez-vous.
- **Gestion des annulations tardives** : mise en place d'un délai minimal d'annulation.

- **Intégration avec d'autres systèmes de santé** : interconnexion avec les systèmes d'information hospitaliers existants au sein du Ministère de la Santé.

En conclusion, ce stage a été une étape déterminante dans mon parcours de formation. Il m'a permis de réaliser que les compétences acquises à l'ISTA HAY SALAM sont directement applicables dans un contexte professionnel réel, et m'a ouvert de nouvelles perspectives sur le métier de développeur web au service de la santé publique.

10. BIBLIOGRAPHIE ET WEBOGRAPHIE

Documentation technique officielle

- **Laravel** — Documentation officielle du framework PHP Laravel :
<https://laravel.com/docs/11.x>
- **Laravel Sanctum** — Documentation de l'authentification API :
<https://laravel.com/docs/11.x/sanctum>
- **React.js** — Documentation officielle de la bibliothèque React :
<https://react.dev>
- **Redux Toolkit** — Documentation de Redux Toolkit :
<https://redux-toolkit.js.org>
- **React Router DOM** — Documentation de la navigation React :
<https://reactrouter.com/en/main>
- **Vite** — Documentation du bundler Vite :
<https://vitejs.dev>
- **MySQL** — Documentation du SGBD MySQL :
<https://dev.mysql.com/doc>

Ressources institutionnelles

- **Ministère de la Santé et de la Protection Sociale du Maroc** :
<https://www.sante.gov.ma>

- **Direction de l'Épidémiologie et de Lutte Contre les Maladies (DELM) :**
<https://www.sante.gov.ma/Pages/Ministere.aspx?IdRub=13>
-

11. ANNEXES

Annexe A — Extrait du code : AuthController.php (Backend)

```
<?php
namespace App\Http\Controllers;

use App\Models\User;
use App\Models\Patient;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Auth;

class AuthController extends Controller
{
    public function register(Request $request) {
        $user = User::create([
            'nom'      => $request->nom,
            'prenom'   => $request->prenom,
            'email'     => $request->email,
            'password' => Hash::make($request->password),
            'role'      => 'patient',
        ]);

        $patient = Patient::create([
            'user_id' => $user->id,
            'telephone' => $request->telephone,
        ]);

        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json(['user' => $user, 'token' => $token]);
    }

    public function login(Request $request) {
        if (!Auth::attempt(['email' => $request->email, 'password' => $request->password])) {
            return response()->json(['message' => "Email ou mot de passe incorrect"], 401);
        }
        $user = Auth::user();
        $token = $user->createToken('auth_token')->plainTextToken;
        return response()->json(['user' => $user, 'token' => $token]);
    }

    public function logout(Request $request) {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Déconnecté avec succès']);
    }
}
```

```
}  
}
```

Annexe B — Extrait du code : RendezVousController.php (Backend)

```
public function prendre(Request $request) {  
    $patient = $request->user()->patient;  
    $creneau = Creneau::find($request->creneau_id);  
  
    if (!$creneau) {  
        return response()->json(['message' => "Le créneau n'existe pas"], 404);  
    }  
    if (!$creneau->disponible) {  
        return response()->json(['message' => "Le créneau est indisponible"], 400);  
    }  
  
    $rdv = RendezVous::create([  
        'patient_id' => $patient->id,  
        'service_id' => $request->service_id,  
        'creneau_id' => $creneau->id,  
        'date'       => $creneau->date,  
        'heure'      => $creneau->heure_debut,  
        'status'     => 'en_attente',  
    ]);  
  
    $creneau->update(['disponible' => false]);  
  
    return response()->json([  
        'message'      => 'Rendez-vous pris avec succès',  
        'rendez_vous' => $rdv,  
    ]);  
}
```

Annexe C — Extrait du code : App.jsx avec routes protégées (Frontend)

```
import { Routes, Route, Navigate } from "react-router-dom"
import { useSelector } from 'react-redux'
import ProtectedRoutes from "../Routes/protectedRoutes"

function App() {
  return (
    <Routes>
      <Route path="/register" element={<Register />} />
      <Route path="/login" element={<Login />} />

      <Route path="/dashboard" element={
        <ProtectedRoutes role="patient">
          <Dashboard />
        </ProtectedRoutes>
      } />

      <Route path="/dashboard/medecin" element={
        <ProtectedRoutes role="medecin">
          <Medecin />
        </ProtectedRoutes>
      } />

      <Route path="/dashboard/secretaire" element={
        <ProtectedRoutes role="secretaire">
          <Secretaire />
        </ProtectedRoutes>
      } />

      <Route path="*" element={<Navigate to="/login" />} />
    </Routes>
  )
}
```

Rapport de stage réalisé par **Milad DLOU** — ISTA HAY SALAM, filière Développement Digital, 2ème année
— Année scolaire 2025–2026

Stage effectué du 01/03/2026 au 31/03/2026 à la Direction de l'Épidémiologie et de Lutte Contre les
Maladies (DELM), Rabat — sous la supervision de M. Haroun